

PEARL: Parameter-Efficient Adaptation with Retrieval-Augmented Learning

Code for the manuscript:

PEARL: Parameter-Efficient Adaptation with Retrieval-Augmented Learning for Molecular Property Prediction

PEARL systematically compares **four experimental settings** for predicting molecular properties on four benchmark datasets (BACE, BBBP, ClinTox, Flavor):

Setting	Description
E2E LoRA	End-to-end LoRA finetuning of chemical language models (CLMs); task head trained jointly with adapters
FT Embed	Optuna-tuned XGB/LGBM/CatBoost classifiers on CLS-token embeddings extracted from the finetuned CLM
FT Embed+PC	Same as FT Embed, but the embedding is concatenated with 473-dim physicochemical (PC) features (148 RDKit descriptors, 30 graph features, 128-bit Morgan fingerprints, 167-bit MACCS keys)
RAFE	FT Embed+PC further augmented with ~73-dim ZINC-250k chemical-neighbourhood features via FAISS retrieval (Retrieval-Augmented Feature Enhancement)

The progression E2E LoRA → FT Embed → FT Embed+PC → RAFE forms an ablation ladder that isolates each component's contribution: finetuning quality, physicochemical feature engineering, and retrieval-based neighbourhood signal from the 249,455-molecule ZINC-250k knowledge base.

Repository Structure

PEARL/	
├─ manuscript/	LaTeX source for the PEARL paper
├─ data/	
│ ├─ raw/	Raw train/valid/test splits (BACE, BBBP, Cli
│ ├─ clean/	Cleaned SMILES splits (generated by Step 1)
│ ├─ zinc250k/	ZINC-250k reference library (SMILES + logP/Q
│ ├─ finetuned_embeddings/	LoRA model embeddings – Step 3 output
│ ├─ finetuned_pc_embeddings/	Embeddings merged with PC features
│ ├─ conformers/	3D conformers for Uni-Mol (generated by Step
│ └─ rag_features/	RAFE features from ZINC-250k retrieval (gene
├─ scripts/	
│ ├─ data_curation/	Step 1 – Data cleaning
│ ├─ smiles/	
│ │ ├─ finetuning/	Step 2–3 – LoRA finetuning + embedding extra
│ │ ├─ ml/	Step 4–5 – FT Embed modelling + RAFE feature
│ │ └─ rag/	Step 5a–5d – RAFE pipeline (ZINC-250k indexi
│ └─ unimol/	
│ ├─ finetuning/	Step 2–3 – Uni-Mol LoRA finetuning + conform
│ ├─ ml/	Step 4 – FT Embed modelling on Uni-Mol emb
│ └─ rag/	Step 5a–5d – Uni-Mol RAFE pipeline
├─ bash/	SLURM job scripts
│ ├─ smiles/	
│ └─ unimol/	
└─ results/	
│ ├─ ft_embeddings/	FT Embed results (XGB/LGBM/CatBoost on finet
│ ├─ rag/	SMILES RAFE results
│ ├─ rag_unimol/	Uni-Mol RAFE results
│ └─ unimol_finetuning/	Uni-Mol E2E LoRA evaluation metrics

Scripts

Data Curation (scripts/data_curation/)

Script	Purpose
data_cleaning.py	Clean raw SMILES for BBBP, ClinTox, Flavor
bace_data_cleaning.py	BACE-specific SMILES cleaning

Setting 1 — E2E LoRA Finetuning

SMILES Track (scripts/smiles/finetuning/)

Script	Purpose
finetune_{bace bbbp clintox flavor}_fl.py	LoRA finetuning with focal loss (4 scripts)
finetune_{bace bbbp clintox flavor}_wl.py	LoRA finetuning with weighted loss (4 scripts)

finetuned_model_embeddings.py	Extract embeddings from all finetuned CLMs
-------------------------------	--

Uni-Mol Track (scripts/unimol/finetuning/)

Script	Purpose
smiles_to_conformers.py	Generate ETKDGV3 + MMFF94 3D conformers
unimol_lora_trainer.py	Shared LoRA trainer utilities
finetune_unimol_{dataset}_{fl wl}.py	Uni-Mol LoRA finetuning (8 scripts)
unimol_embeddings.py	Extract 2560-dim Uni-Mol embeddings

Settings 2 & 3 — FT Embed and FT Embed+PC (ML on Finetuned Embeddings)

Feature Preparation (scripts/smiles/ml/)

Script	Purpose
bace_feature_extraction_ft_model.py	Merge BACE finetuned embeddings + 473-dim PC features
pc_feature_extraction_ft_model_refactored.py	Same for BBBP, ClinTox, Flavor

SMILES FT Embed Modelling (scripts/smiles/ml/)

Script	Purpose
bace_modelling_refactored.py	Optuna-tuned XGB/LGBM/CatBoost on BACE finetuned embeddings
bbbp_modelling_refactored.py	Same for BBBP
clintox_modelling_refactored.py	Same for ClinTox
flavor_modelling_refactored.py	Same for Flavor
bace_pc_modelling_refactored.py	Same for BACE embeddings + PC features
bbbp_pc_modelling_refactored.py	Same for BBBP
clintox_pc_modelling_refactored.py	Same for ClinTox
flavor_pc_modelling_refactored.py	Same for Flavor

aggregate_metrics.py	Aggregate per-dataset FT Embed metrics
aggregate_all_metrics.py	Aggregate across all datasets

Uni-Mol FT Embed Modelling (scripts/unimol/ml/)

Script	Purpose
unimol_modelling.py	XGB/LGBM/CatBoost on Uni-Mol LoRA embeddings (all datasets)

Setting 4 — RAFE (Retrieval-Augmented Feature Enhancement)

SMILES RAFE Pipeline (scripts/smiles/rag/)

Script	Purpose
embed_zinc250k.py	Embed all 249,455 ZINC-250k molecules with finetuned CLMs
build_zinc_index.py	Build GPU FAISS FlatIP indices from ZINC-250k embeddings
rag_feature_extraction.py	Query FAISS indices; compute RAFE features per molecule
rag_modelling_{bace bbbp clintox flavor}.py	Train Optuna-tuned XGB/LGBM/CatBoost (4 scripts)
aggregate_rag_results.py	Consolidate metrics across all models and datasets

Uni-Mol RAFE Pipeline (scripts/unimol/rag/)

Script	Purpose
embed_zinc250k_unimol.py	Embed ZINC-250k with finetuned Uni-Mol models
build_zinc_index_unimol.py	Build Uni-Mol FAISS indices
rag_feature_extraction_unimol.py	Extract Uni-Mol RAFE features
rag_modelling_{dataset}_unimol.py	Uni-Mol RAFE classifiers (4 scripts)

Large Files (not in repo)

Finetuned model checkpoints, ZINC embeddings, and FAISS indices (~75 GB total) are stored externally. Set this variable before running any script:

```
export PEARL_EXTRAS="/export/cse/rmall/Raghvendra/EffiChem_Extras"
```

Requirements

```
pip install -r requirements.txt
```

faiss-gpu requires a CUDA GPU. On CPU-only machines use faiss-cpu instead. unimol-tools is only needed for the Uni-Mol track.

End-to-End Workflow

Step 1	Data cleaning	
Step 2	LoRA finetuning	├── SMILES track (ChemBERTa, MolFc)
		└── Uni-Mol track (conformers → fi
Step 3	Extract finetuned embeddings	
Step 4	Prepare ML input	├── FT Embed: embeddings only
		├── FT Embed: embeddings + PC feat
		└── FT Embed: Uni-Mol embeddings (
Step 4b	Train FT Embed classifiers	(XGB/LGBM/CatBoost, Optuna-tuned)
Step 5	RAFE pipeline	├── 5a Embed ZINC-250k
		├── 5b Build FAISS indices
		├── 5c Extract RAFE features
		└── 5d RAG modelling

Step 1 — Data Cleaning

```
python scripts/data_curation/data_cleaning.py           # BBBP, ClinTox
python scripts/data_curation/bace_data_cleaning.py      # BACE
```

Output: data/clean/{dataset}_datasets/{split}_clean.csv

Step 2 — LoRA Finetuning [Setting 1: E2E LoRA]

SMILES track — requires a .env file at the repo root with WANDB_API_KEY=<key>.

```

# Focal loss (best for BACE)
sbatch bash/smiles/run_bace_finetuning_fl.sh
sbatch bash/smiles/run_bbbp_finetuning_fl.sh
sbatch bash/smiles/run_clintox_finetuning_fl.sh
sbatch bash/smiles/run_flavor_finetuning_fl.sh

# Weighted loss (best for BBBP, ClinTox, Flavor)
sbatch bash/smiles/run_bace_finetuning_wl.sh
sbatch bash/smiles/run_bbbp_finetuning_wl.sh
sbatch bash/smiles/run_clintox_finetuning_wl.sh
sbatch bash/smiles/run_flavor_finetuning_wl.sh

```

Uni-Mol track — generate conformers first.

```

sbatch bash/unimol/run_conformer_gen.sh
# or: python scripts/unimol/finetuning/smiles_to_conformers.py --data

sbatch bash/unimol/run_unimol_bace_fl.sh    # repeat for _wl, bbbp, c

```

Output:

```

$PEARL_EXTRAS/{focal_loss|weighted_loss}_{DATASET}/{model}_LoRA_Finetuned/

```

Step 3 — Extract Finetuned Embeddings

```

# SMILES track
sbatch bash/smiles/run_embedding_extraction.sh
# or: python scripts/smiles/finetuning/finetuned_model_embeddings.py

# Uni-Mol track
sbatch bash/unimol/run_unimol_embeddings.sh
# or: python scripts/unimol/finetuning/unimol_embeddings.py

```

Output (SMILES):

```

data/finetuned_embeddings/{Dataset}_Embeddings/{prefix}_{split}_embed.

```

csv Output (Uni-Mol):

```

$PEARL_EXTRAS/unimol_embeddings/{Dataset}_Embeddings/{prefix}_{split}_embed.csv

```

Step 4 — Prepare FT Embed Input

Merges finetuned embeddings with 473-dimensional physicochemical features.

```

python scripts/smiles/ml/bace_feature_extraction_ft_model.py
python scripts/smiles/ml/pc_feature_extraction_ft_model_refactored.py

```

Output:

```
data/finetuned_pc_embeddings/{Dataset}_Embeddings/{prefix}_{split}_features.csv
```

Step 4b — FT Embed Modelling [Settings 2 & 3: FT Embed / FT Embed+PC]

Trains Optuna-tuned XGBoost / LightGBM / CatBoost classifiers on finetuned embeddings.

SMILES FT Embed (embeddings only):

```
sbatch bash/smiles/run_bace_ft.sh
sbatch bash/smiles/run_bbbp_ft.sh
sbatch bash/smiles/run_clintox_ft.sh
sbatch bash/smiles/run_flavor_ft.sh
```

SMILES FT Embed + PC features:

```
sbatch bash/smiles/run_bace_pc_ft.sh
sbatch bash/smiles/run_bbbp_pc_ft.sh
sbatch bash/smiles/run_clintox_pc_ft.sh
sbatch bash/smiles/run_flavor_pc_ft.sh
```

Uni-Mol FT Embed:

```
sbatch bash/unimol/run_unimol_ft_modelling.sh
# or: python scripts/unimol/ml/unimol_modelling.py --dataset all --cc
```

Output:

```
results/ft_embeddings/{Dataset}_FT_Results/{model}/{split}/metrics.json
```

Step 5a — Embed ZINC-250k

```
# SMILES track
sbatch bash/smiles/run_embed_zinc_bace.sh    # repeat for bbbp, clintox
# or: python scripts/smiles/rag/embed_zinc250k.py --dataset {bace|bbbp|clintox}

# Uni-Mol track
sbatch bash/unimol/run_embed_zinc_unimol_bace.sh    # repeat for bbbp, clintox
# or: python scripts/unimol/rag/embed_zinc250k_unimol.py --dataset {bace|bbbp|clintox}
```

Output: \$PEARL_EXTRAS/zinc_embeddings/{dataset}/{col_name}.npz

Step 5b — Build FAISS Indices

```
# SMILES track
sbatch bash/smiles/run_build_zinc_index.sh
# or: python scripts/smiles/rag/build_zinc_index.py --dataset bace --

# Uni-Mol track
sbatch bash/unimol/run_build_zinc_index_unimol.sh
# or: python scripts/unimol/rag/build_zinc_index_unimol.py --dataset
```

Output: \$PEARL_EXTRAS/rag_indices/{dataset}/{col_name}.index

Step 5c — Extract RAFE Features

```
# SMILES track
sbatch bash/smiles/run_rag_features.sh
# or: python scripts/smiles/rag/rag_feature_extraction.py --dataset {

# Uni-Mol track
sbatch bash/unimol/run_rag_features_unimol.sh
# or: python scripts/unimol/rag/rag_feature_extraction_unimol.py --dataset
```

Output: data/rag_features/{dataset}/{col_name}_{split}_rag.csv

Step 5d — RAFE Modelling [Setting 4: RAFE]

```
# SMILES track
sbatch bash/smiles/run_rag_modelling_bace.sh
sbatch bash/smiles/run_rag_modelling_bbbp.sh
sbatch bash/smiles/run_rag_modelling_clintox.sh
sbatch bash/smiles/run_rag_modelling_flavor.sh

# Uni-Mol track
sbatch bash/unimol/run_rag_modelling_unimol_bace.sh
sbatch bash/unimol/run_rag_modelling_unimol_bbbp.sh
sbatch bash/unimol/run_rag_modelling_unimol_clintox.sh
sbatch bash/unimol/run_rag_modelling_unimol_flavor.sh
```

Output: results/rag/{dataset}/{col_name}/metrics.json
results/rag_unimol/{dataset}/{col_name}/metrics.json

Environment Variables

Variable	Default	Purpose
----------	---------	---------

PEARL_EXTRAS	/export/cse/rmall/Raghvendra/EffiChem_Extras	Root for large external files (~75 GB)
WANDB_API_KEY	<i>(none — set in .env)</i>	WandB sweep key for LoRA hyperparameter tuning
RANDOM_SEED	42	Global random seed
N_JOBS	half of available CPUs	Parallel jobs for tree-based classifiers
OPTUNA_TRIALS	20	Optuna hyperparameter trials per model

Citation

If you use this code or data, please cite the PEARL manuscript.

© Raghvendra Mall. All rights reserved.